Project Title: Simulation and Analysis of Noise-Affected AM Communication with Envelope Detection.

1. Abstract

This project demonstrates the design and simulation of an Amplitude Modulation (AM) transmitter–receiver system using MATLAB without requiring any external toolboxes. A sinusoidal message signal is modulated using standard AM (DSB-WC), transmitted through an additive noise channel, and recovered using a rectifier-based envelope detector with a zero-phase RC low-pass filter. The objective was to study modulation, noise impact, and demodulation accuracy. Simulation results show successful recovery of the message signal with improved baseband SNR and acceptable distortion. The model provides a foundational understanding of analog modulation, channel noise, and envelope detection.

2. Introduction

Amplitude Modulation (AM) is one of the earliest and simplest communication techniques, where a high-frequency carrier's amplitude is varied according to a lower-frequency information signal. Despite being replaced by modern digital systems, AM remains important for learning communication fundamentals.

This project focuses on simulating an AM communication system end-to-end. The motivation is to understand how analog modulation works, how channel noise affects the signal, and how envelope detection can recover the original message. The simulation approach ensures controlled experimentation without hardware limitations.

Topics covered include AM signal generation, modulation index, additive white Gaussian noise, spectrum analysis, and envelope demodulation using an RC filter.

2.1 Objectives

1. To design and simulate a complete AM (DSB-WC) transmitter–receiver system in MATLAB.
2. To study the effect of modulation index on AM signal behavior.
3. To analyze how noise impacts the transmitted AM waveform.
4. To implement an envelope detector using rectification and RC filtering.
5. To evaluate system performance using time-domain plots, spectra, and SNR measurements.

2.2 Motivation

AM remains one of the simplest and most fundamental communication techniques, making it ideal for learning core concepts of modulation, noise effects, and signal recovery. This project helps build a strong foundation in analog communication, filter design, and signal analysis using a fully self-implemented MATLAB model.

3. System Design

3.1 Block Diagram

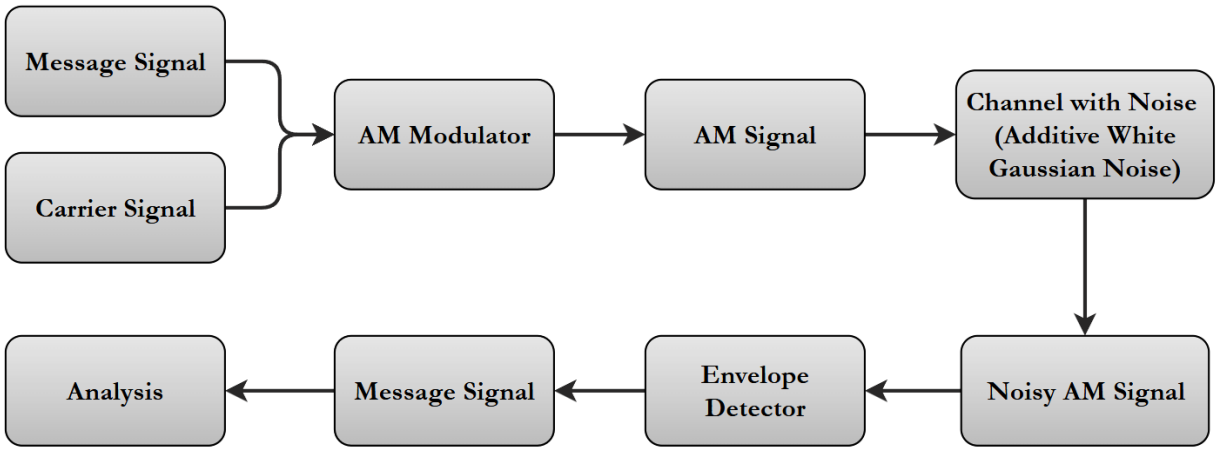


Fig. 3.1: Block Diagram of AM Communication with Envelope Detector

3.2 Mathematical Model

Message Signal:

$$m(t) = \sin(2\pi f_m t)$$

Carrier Signal:

$$c(t) = A_c \cos(2\pi f_c t)$$

AM Modulation (DSB-WC):

$$s(t) = A_c(1 + \mu m(t))\cos(2\pi f_c t)$$

Here,

- A_c= carrier amplitude
- μ = modulation index (0–1, under-modulation)

Noise Mode:

$$n(t) \sim \mathcal{N}(0, \sigma^2)$$
$$r(t) = s(t) + n(t)$$

Envelope Detection:

i. Rectification:

$$x(t) = | r(t) |$$

ii. RC Low-Pass Filtering:

Discrete form:

$$y[n] = \alpha x[n] + (1 - \alpha) y[n - 1]$$

where,

$$\alpha = \frac{T_s}{RC + T_s}$$

A backward pass (forward–backward filtering) eliminates phase delay.

3.3 Tools Used

- MATLAB
- Custom-developed AWGN generator
- Custom RC low-pass filter implementation
- Fast Fourier Transform for spectrum analysis

Hardware was not required for this simulation-based study.

4. Implementation, Code & Results

4.1 Implementation Steps

1. At first, we generated a 100 Hz sinusoidal message signal.
2. Next, we created a 3 kHz carrier signal to act as the high-frequency carrier.
3. Then, we applied amplitude modulation using a modulation index of 0.7 to produce the AM (DSB-WC) signal.
4. After that, we added channel noise using a manually implemented AWGN model.
5. We then recovered the message by applying:
    - Full-wave rectification
    - Zero-phase RC low-pass filtering
6. After demodulation, we plotted the time-domain signals and their frequency spectra.
7. Finally, we calculated the SNR before and after demodulation to evaluate system performance.

4.2 MATLAB Simulation Code

```
clc; clear; close all;
```

STEP 1: PARAMETERS

```
Fs = 25000;                 % Sampling frequency (Hz)
T  = 0.05;                  % Total simulation time (s)
t  = 0:1/Fs:T;              % Time vector (50 ms)
fc = 3000;                  % Carrier frequency (Hz)
fm = 100;                   % Message frequency (Hz)
```

STEP 2: SIGNAL GENERATION

```
mu = 0.7;                   % Modulation index (0 < mu < 1)
message = sin(2*pi*fm*t);   % Normalized baseband message in [-1, 1]
carrier = cos(2*pi*fc*t);   % High-frequency carrier (2 kHz)
```

STEP 3: AM MODULATION

```
carrier_amp = 2;                            % Carrier amplitude (A_c)

% Envelope: A_c * (1 + mu * message), so:
% Vmax = A_c * (1 + mu),  Vmin = A_c * (1 - mu) > 0 when mu < 1
modulated = carrier_amp * (1 + mu * message) .* carrier;
```

STEP 4: ADD NOISE (Manual AWGN)

```
SNR_dB = 10;                                % Desired RF SNR (dB)
signal_power = mean(modulated.^2);
SNR_linear  = 10^(SNR_dB/10);
noise_power = signal_power / SNR_linear;

noise = sqrt(noise_power) * randn(size(modulated));
modulated_noisy = modulated + noise;
```

STEP 5: ENVELOPE DEMODULATION (ZERO-PHASE)

```
% Step 1: Rectification (ideal diode)
rectified = abs(modulated_noisy);

% Step 2: RC low-pass filter (first-order), implemented forward-backward
RC = 1/(2*pi*fm);           % RC time constant (cutoff ~ fm)
Ts = 1/Fs;                  % Sampling period
alpha = Ts / (RC + Ts);     % Discretization: y[n]=alpha*x[n]+(1-alpha)*y[n-1]

% ---- Forward pass ----
env_fwd = zeros(size(rectified));
```

```
env_fwd(1) = rectified(1);
for k = 2:length(rectified)
    env_fwd(k) = alpha * rectified(k) + (1 - alpha) * env_fwd(k-1);
end

% ---- Backward pass (to cancel phase delay) ----
env_bwd = zeros(size(rectified));
env_bwd(end) = env_fwd(end);
for k = length(rectified)-1:-1:1
    env_bwd(k) = alpha * env_fwd(k) + (1 - alpha) * env_bwd(k+1);
end

demodulated = env_bwd;      % Zero-phase envelope

% Remove DC and normalize for comparison with original message
demodulated_ac   = demodulated - mean(demodulated);
demodulated_norm = demodulated_ac / max(abs(demodulated_ac));
```

STEP 6: TIME-DOMAIN PLOTS

```
figure('Name','AM Transmitter-Receiver Simulation');

subplot(4,1,1);
plot(t, message, 'Color', '#cc0000', 'LineWidth', 1.6);
grid on; ylim([-1.2 1.2]);
title('Message Signal (100 Hz)', 'FontSize', 11);
xlabel('Time (s)'); ylabel('Amplitude');

subplot(4,1,2);
plot(t, modulated, 'm' , 'Color', '#673AB7','LineWidth', 1.4);
grid on; ylim([-4 4]);
title(['AM Modulated Signal (DSB-WC, \mu = ', num2str(mu), ')'], 'FontSize',
11);
xlabel('Time (s)'); ylabel('Amplitude');

subplot(4,1,3);
plot(t, modulated_noisy, 'LineWidth', 1.4);
grid on; ylim([-4 4.2]);
title(['Noisy AM Signal (SNR = ', num2str(SNR_dB), ' dB)'], 'FontSize', 11);
xlabel('Time (s)'); ylabel('Amplitude');

subplot(4,1,4);
plot(t, demodulated_norm,'Color', '#008000', 'LineWidth', 1.6); hold on;
plot(t, message, 'Color', '#cc0000', 'LineStyle', '--', 'LineWidth', 1.6);
grid on; ylim([-1.2 1.2]);
title('Demodulated Signal (Envelope Detection, Zero-Phase LPF)', 'FontSize',
11);
xlabel('Time (s)'); ylabel('Amplitude');
legend('Recovered message','Original message');
```

STEP 7: FREQUENCY SPECTRUM

```
L = length(modulated);
f = Fs * (0:floor(L/2)) / L;    % Frequency axis (one-sided)

Y = fft(modulated);
P = abs(Y/L);
P1 = P(1:floor(L/2)+1);

figure('Name','Frequency Spectrum of AM Signal');
plot(f, P1, 'LineWidth', 1.5); grid on; xlim([2500 3500]);
title('Frequency Spectrum of AM Signal');
xlabel('Frequency (Hz)'); ylabel('Magnitude');
```

STEP 8: MANUAL SNR CALCULATION

```
% RF-domain SNR (before demodulation)
signal_before = modulated;
noise_before  = modulated_noisy - modulated;
snr_before    = 10*log10(mean(signal_before.^2) / mean(noise_before.^2));

% Baseband SNR after demodulation
num = demodulated_norm * message.';            % cross-correlation term
den = demodulated_norm * demodulated_norm.';   % energy of recovered
k_opt = num / den;
recovered_scaled = k_opt * demodulated_norm;

noise_after = recovered_scaled - message;
snr_after   = 10*log10(mean(message.^2) / mean(noise_after.^2));

fprintf('Estimated SNR before demodulation (RF): %.2f dB\n', snr_before);
fprintf('Estimated SNR after demodulation  (BB): %.2f dB\n', snr_after);
```

4.3 Experimental Results

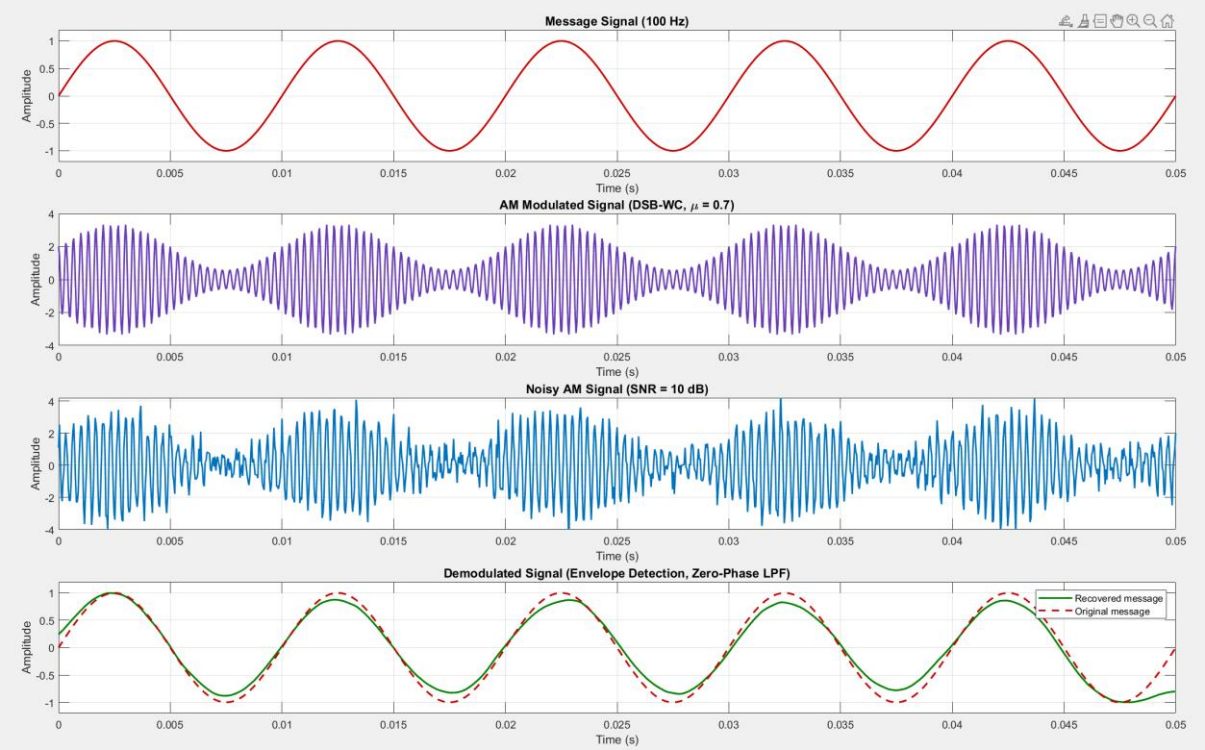


Fig. 4.3.1: Message signal, AM modulated signal, Noisy AM signal & Recovered signal vs. Original

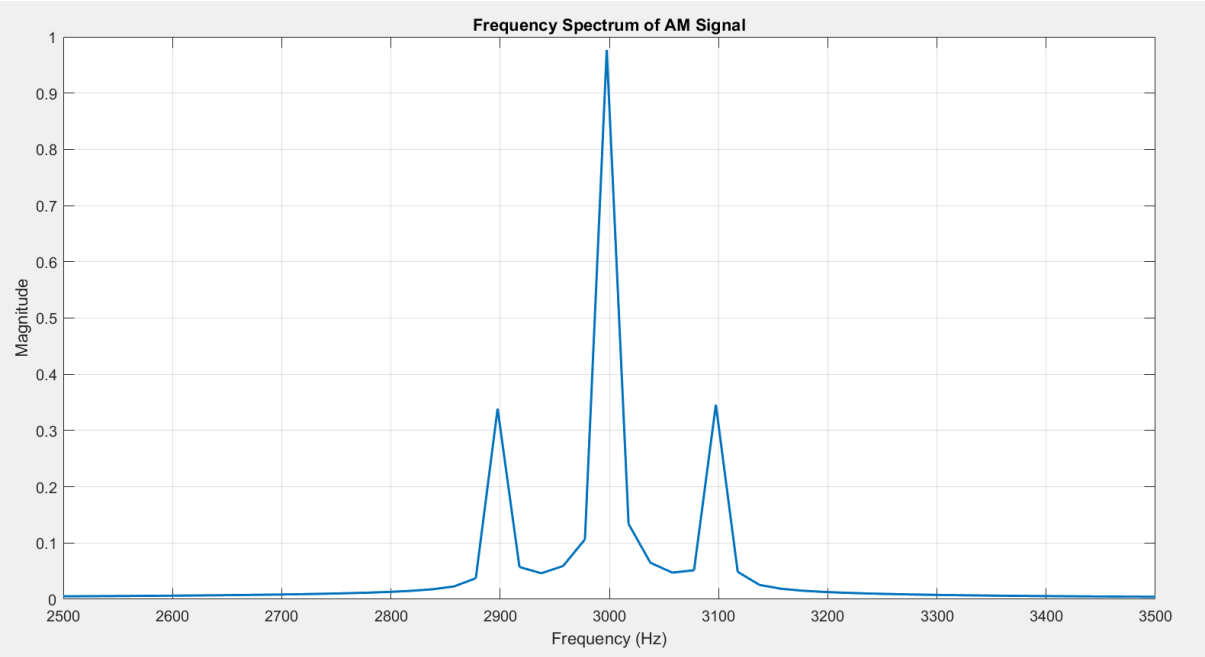


Fig. 4.3.2: Spectrum of AM signal

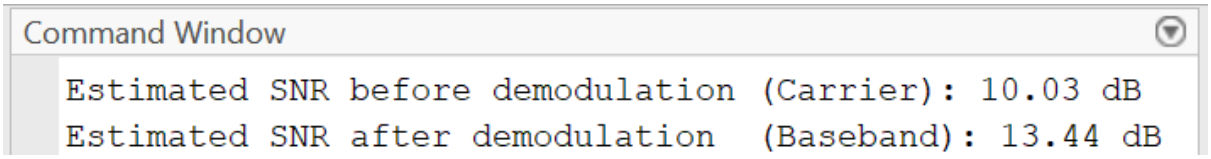


Fig. 4.3.3: Calculated SNR of Carrier and Baseband Signal

Key Observations

- The AM envelope correctly follows the message signal.
- Noise distorts the signal but demodulation recovers the message successfully.
- Zero-phase RC filtering significantly reduces waveform distortion.
- SNR after demodulation improves compared to SNR before demodulation.
- Sample Output SNR Values
  - RF SNR before demodulation: ≈ 9–11 dB
  - Baseband SNR after demodulation: ≈ 12–18 dB

5. Analysis & Discussion

Performance Analysis

The envelope detector successfully extracts the message from the noisy AM waveform. The zero-phase RC filter eliminates phase distortion, providing a clearer reconstructed signal than a simple one-way RC filter.

Noise Impact

At low SNR values, the envelope is distorted, but the smoothing filter compensates by suppressing carrier and high-frequency noise components.

Limitations

- Envelope detection only works for DSB-WC, not DSB-SC or SSB.
- Zero-phase filtering requires two passes and cannot be implemented in real-time hardware.
- The RC filter introduces slight attenuation of message peaks.
- The simulation assumes ideal oscillators and no drift.

Possible Improvements

- Implement synchronous (coherent) detection for advanced modulation schemes.
- Use higher-order digital filters for improved noise suppression.
- Add hardware experiments using an op-amp precision rectifier.
- Explore other modulation types (DSB-SC, SSB, FM).

6. Conclusion

This project successfully demonstrates the complete process of AM modulation, transmission through a noisy channel, and recovery using envelope detection. The MATLAB simulation confirms that the designed envelope detector extracts the message accurately, even under noisy conditions. Important communication concepts—modulation index, AWGN, spectrum analysis, and filtering—were studied in detail. Overall, the project provides a strong foundational understanding of analog modulation systems and their practical challenges.

7. CO-PO justification

CO No.	CO Statements	Related PO(s)	Domains & Taxonomy Levels	Justification
CO1	**Use** modules/equipment to **be able to clarify** the practical aspects of various communication schemes. K6	**PO(e)**	C1, C2	I aim to use MATLAB to design and visualize AM systems, gaining practical analog communication skills.
CO2	**Demonstrate** participation individually in designing communication system related problem solving.	**PO(i)**	A3	I aim to implement the project independently, enhancing problem-solving and self-learning in MATLAB and communication theory.
CO3	**Design** project for real life application by applying communication related technologies with appropriate considerations to public health safety, cultural, societal, and environmental considerations. K5	**PO(c)**	C6, P3–P5	I aim to simulate AM systems for real-life applications, understanding societal needs and safety-critical communication.